

Trabajo práctico N°6



Integrantes:

- Medina, Gabriel 14086
- Vasquez, Nicolas 14097
- Albanez, Luciano 13185
- Fattiboni, Álvaro 13201

- 1) Un planificador se fundamenta en algoritmos de búsqueda en grafos o en espacios de estados. Estos algoritmos permiten explorar diferentes secuencias de acciones hasta alcanzar un estado meta que satisfaga las condiciones deseadas.

Entre los más relevantes se encuentran:

- Búsquedas no informadas (o ciegas): como la búsqueda en amplitud (BFS), en profundidad (DFS) o la búsqueda en profundidad iterativa. Estas no emplean conocimiento adicional, por lo que suelen ser menos eficientes.

- Búsquedas informadas (heurísticas): como A* o Greedy Best-First Search, que utilizan funciones heurísticas para guiar la exploración de estados hacia la solución más prometedora.

- Algoritmos especializados de planificación: como Graphplan o SATPlan, que representan de manera compacta el espacio de estados y aplican técnicas de optimización.

En conclusión, los planificadores suelen recurrir principalmente a algoritmos heurísticos ya que permiten encontrar soluciones en menos tiempo y con menor costo computacional.

- 2) En el modelo de planificación STRIPS (Stanford Research Institute Problem Solver), cada acción está definida a través de tres componentes esenciales:

- Precondiciones: Son los hechos o condiciones que deben cumplirse en el estado actual para que la acción pueda ejecutarse. Ejemplo: para realizar la acción abrir puerta, es necesario tener la llave.

- Efectos positivos (add list): Representan los hechos que pasan a ser verdaderos una vez ejecutada la acción. Ejemplo: tras abrir la puerta, el hecho puerta abierta se agrega al estado.
- Efectos negativos (delete list): Indican los hechos que dejan de cumplirse al ejecutar la acción. Ejemplo: al abrir la puerta, el hecho puerta cerrada se elimina del estado.

Este esquema permite modelar de forma sencilla y estructurada los cambios que las acciones producen en el mundo.

- 3) Existen dos enfoques principales en la planificación clásica: la planificación hacia adelante (forward search) y la planificación hacia atrás (backward search). Ambos presentan ventajas y limitaciones que deben considerarse según el problema a resolver.

Planificación hacia adelante:

- Ventajas:
 - El proceso inicia desde el estado inicial, lo que facilita la aplicación práctica y la consistencia con el mundo real.
 - Solo se generan acciones que son ejecutables en el contexto actual.
- Desventajas:
 - Puede explorar una gran cantidad de estados irrelevantes antes de alcanzar la meta.
 - Requiere de buenas heurísticas para ser eficiente, de lo contrario el proceso es costoso.

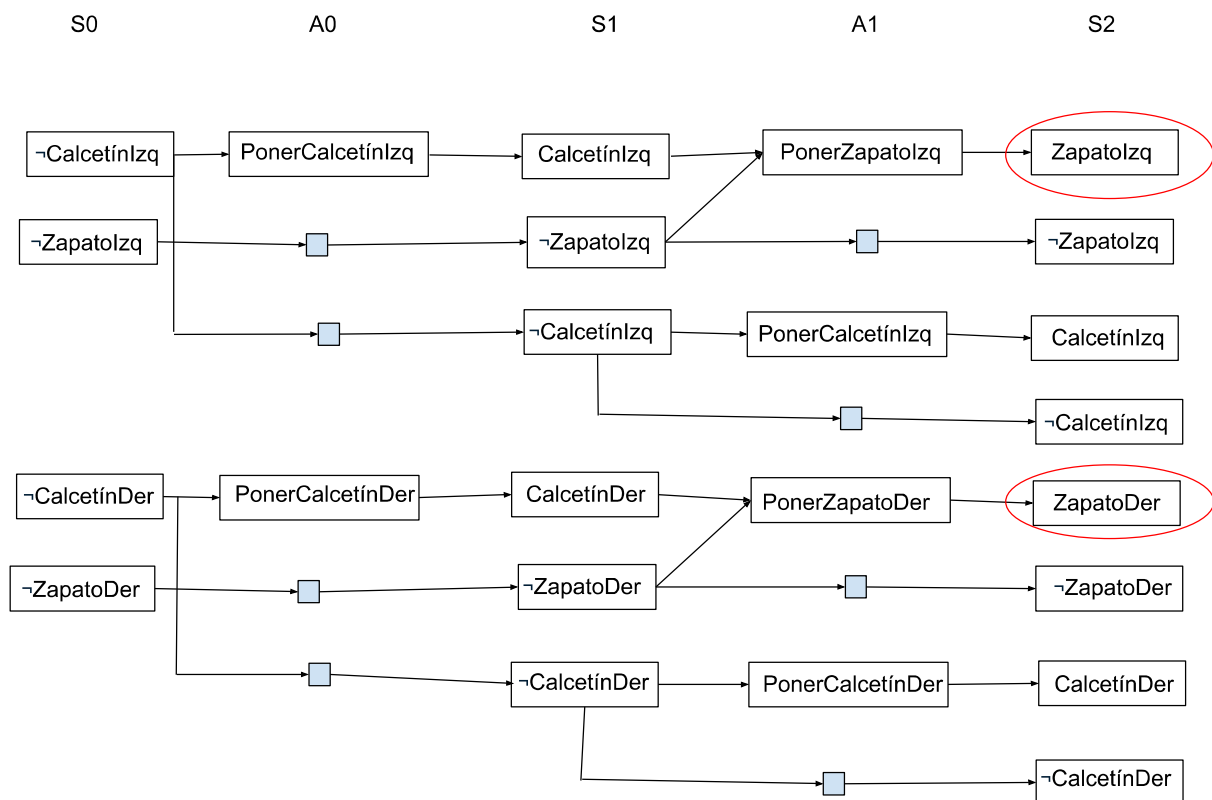
Inteligencia Artificial 2025 - Grupo 5

Planificación hacia atrás:

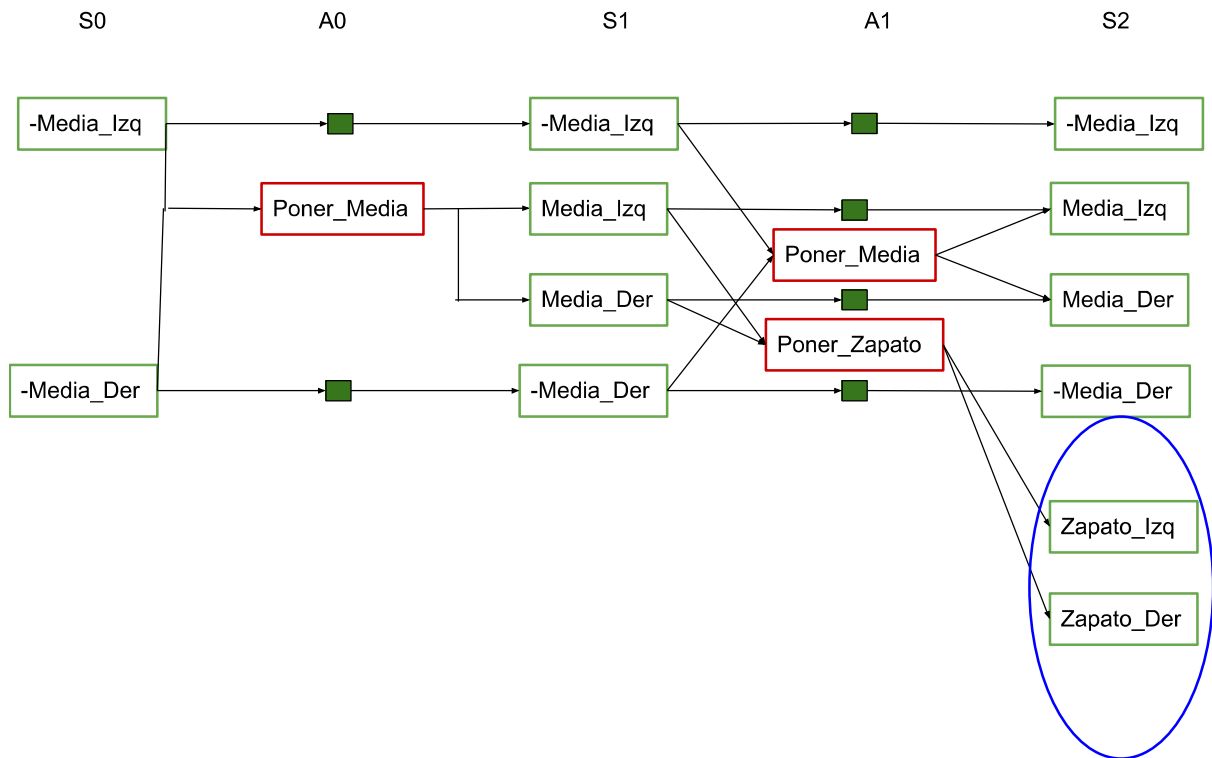
- Ventajas:
 - El proceso comienza desde el objetivo, lo que reduce la exploración de estados innecesarios.
 - Es más eficiente cuando los objetivos están claramente definidos y son pocos.
- Desventajas:
 - Puede ser difícil encontrar las acciones que satisfagan las condiciones del objetivo.
 - Si las metas poseen múltiples precondiciones generales, el proceso se vuelve complejo y costoso.

4)

Forma 1



Forma 2



Plan de orden parcial (solución)

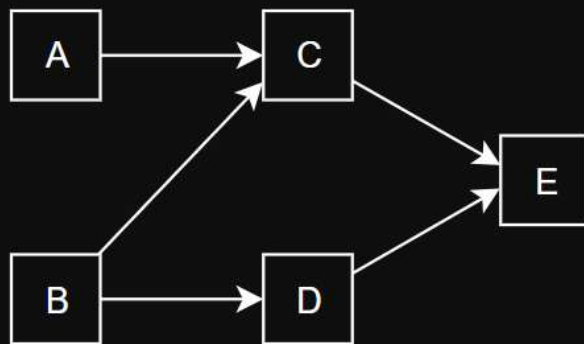
PonerCalcetinIzq → PonerZapatoIzq → PonerCalcetinDer →
PonerZapatoDer

Tendríamos 6 linealizaciones diferentes, 2 serán del tipo que mostramos pero cambiando el orden haciendo primero el derecho y luego el izquierdo. Luego tenemos 4 más en los cuales se permutan el orden primero en las acciones de A0 (PonerCalcetinIzq y PonerCalcetinDer) y luego en A1 (PonerZapatoIzq y PonerZapatoDer) por lo que tendríamos 2^2 soluciones para estos casos, dando así un total de 6.

5)

5.1

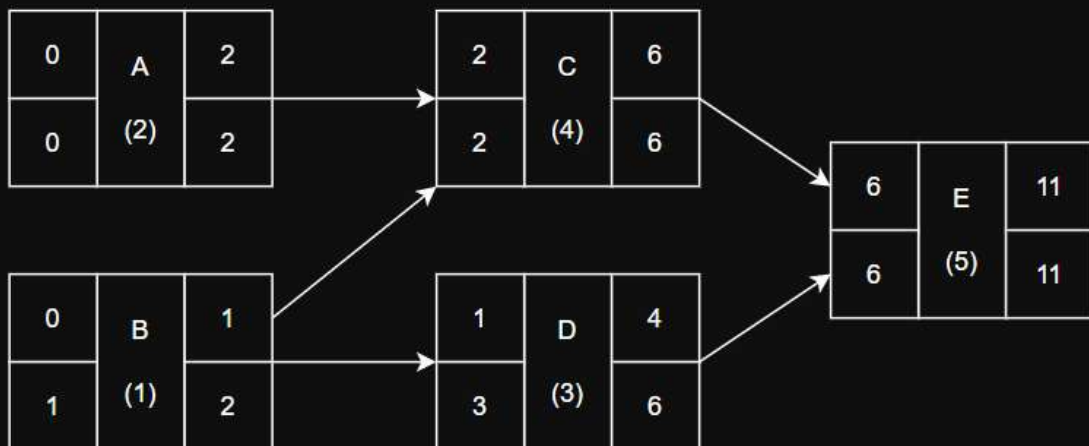
Primero se realizó un diagrama de red bayesiana para determinar los posibles estados iniciales y la dependencia entre los distintos nodos.



A: 2 semanas
B: 1 semana
C: 4 semanas
D: 3 semanas
E: 5 semanas

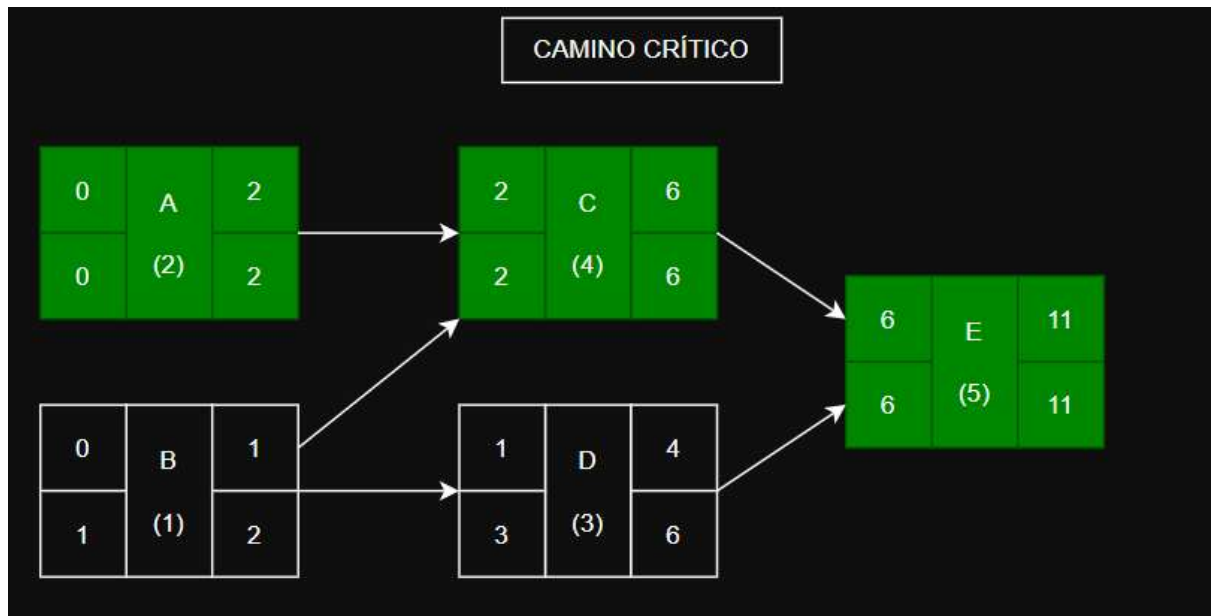
Plan de orden parcial

Ensamblar A → Ensamblar B → Ensamblar C → Ensamblar D → Ensamblar E



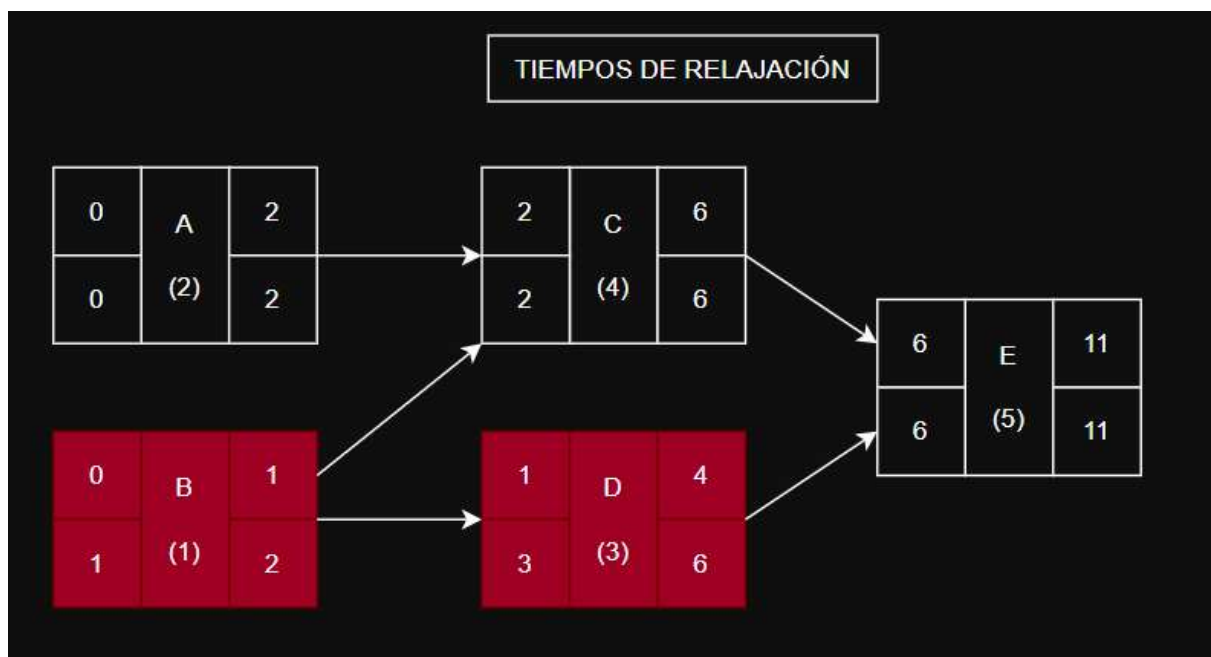
5.2

Camino crítico



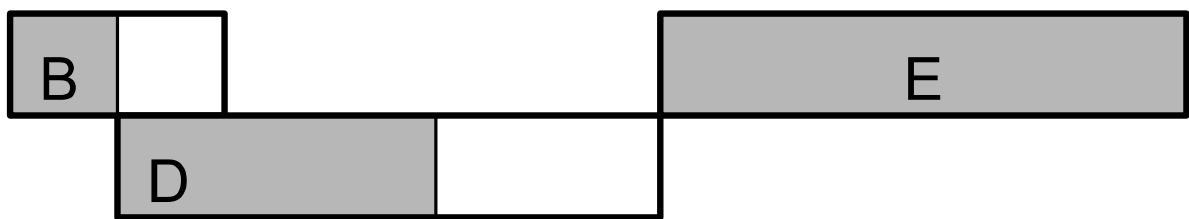
5.3

Tiempos de relajación



5.4

Diagrama temporal



Ejercicio 6

Introducción

En este trabajo se presenta la resolución de un problema de planificación utilizando el lenguaje **PDDL (Planning Domain Definition Language)**. PDDL es un lenguaje inspirado en la formalización STRIPS que permite describir tareas de planificación de manera estandarizada. Su objetivo es especificar dominios de planificación (acciones y condiciones) y problemas concretos (objetos, estado inicial y meta) que luego puedan ser resueltos por un planificador automático.

Breve teoría

STRIPS

STRIPS (Stanford Research Institute Problem Solver) fue uno de los primeros formalismos en planificación automática. Se basa en:

- Un estado inicial del mundo.
- Una descripción de las acciones, cada una con **precondiciones** y **efectos**.
- Una condición objetivo que se quiere alcanzar.

El planificador busca una secuencia de acciones que transforme el estado inicial en un estado donde la condición objetivo se cumpla.

PDDL

PDDL se desarrolló para estandarizar la descripción de problemas de planificación. Es un lenguaje ampliamente utilizado en la investigación en IA y en competencias de planificación.

Componentes de una tarea de planificación en PDDL:

- **Objects (Objetos):** Elementos del mundo que nos interesan.
- **Predicates (Predicados):** Propiedades de los objetos o relaciones entre ellos; pueden ser verdaderas o falsas.
- **Initial state (Estado inicial):** El estado del mundo en el que se comienza.
- **Goal specification (Objetivo):** Lo que queremos que sea verdadero al finalizar.

- **Actions/Operators (Acciones/Operadores):** Formas de cambiar el estado del mundo.

En PDDL, las tareas de planificación se especifican en **dos archivos separados**:

1. **Archivo de dominio:** Contiene los predicados y las acciones.
 2. **Archivo de problema:** Contiene los objetos, el estado inicial y la especificación del objetivo.
-

La consigna

El enunciado plantea un escenario con un robot, una caja, café y una carta, distribuidos en distintos lugares (oficina, cafetería y buzón). El robot debe utilizar la caja para transportar el café y la carta hasta la oficina, cumpliendo las siguientes restricciones:

- El robot solo puede tener la caja una vez la tome.
 - El robot debe moverse entre los lugares para recoger los objetos.
 - Los objetos deben quedar dentro de la caja antes de ser transportados.
-

Lo realizado

1. **Definición del dominio:**
 - Se modelaron los predicados básicos: ubicación del robot, ubicación de objetos, tener la caja, objetos dentro de la caja.

Lugar(x)	Verdadero si x es un lugar
Robot(x)	Verdadero si x es un robot
Objeto(x)	Verdadero si x es un objeto
En-roblugar(x)	Verdadero si x es un lugar y el robot está en x
En-objlugar(x,y)	Verdadero si x es un objeto, y es un lugar; x está en y
Tiene-caja(x)	Verdadero si x es un robot y tiene una caja
En-objcaja(x)	Verdadero si x es un objeto y está dentro de la caja

- Se definieron las acciones principales: *tomar-caja*, *mover*, *poner-objeto-en-caja* y *sacar-objeto-de-caja*.

2. Definición del problema:

- Se especificaron los objetos: lugares, robot y elementos (café, carta, caja).
- Se definió el estado inicial: el robot empieza en la oficina, la caja está en la oficina, el café en la cafetería y la carta en el buzón.

-Lugar(oficina), Lugar(cafetería), Lugar(buzón) son verdaderos.
-Robot(robot) es verdadero.
-Objeto(café), Objeto(caja), Objeto(carta) son verdaderos.
-En-roblugar(oficina) es verdadero.
-En-objlugar(café,cafetería), En-objlugar(carta,buzón), En-objlugar(caja,oficina) son verdaderos.
-Todo lo demás es falso.

- El objetivo: que tanto el café como la carta estén en la oficina.

3. Obtención del plan:

- El planificador generó un plan válido que cumple el objetivo, aunque el orden de recogida de los objetos no coincide

exactamente con la consigna original.

Archivos finales

Dominio

```
(define (domain TP6_EJ6_dom)
  (:requirements :strips)

  (:predicates
    (lugar ?l)
    (robot ?r)
    (objeto ?o)
    (En-roblugar ?l)
    (En-objlugar ?o ?l)
    (Tiene-caja ?r)
    (En-objcaja ?o)
  )

  (:action tomar-caja
    :parameters (?caja ?oficina ?robot)
    :precondition (and (objeto ?caja)
      (En-roblugar ?oficina) (En-objlugar ?caja ?oficina)
      (robot ?robot))
    :effect (and (Tiene-caja ?robot)
      (not(En-objlugar ?caja ?oficina)))
  )

  (:action mover
    :parameters (?desde ?hasta)
    :precondition (and (lugar ?desde) (lugar
      ?hasta) (En-roblugar ?desde))
    :effect (and (En-roblugar ?hasta)
      (not(En-roblugar ?desde)))
  )
```

```
(:action poner-objeto-en-caja
  :parameters (?obj ?lugar ?robot)
  :precondition (and(En-objlugar ?obj ?lugar)
(Tiene-caja ?robot) (En-roblugar ?lugar))
  :effect (and (En-objcaja ?obj)
(not(En-objlugar ?obj ?lugar)))
)
```

```
(:action sacar-objeto-de-caja
  :parameters (?obj ?lugar ?robot)
  :precondition (and (En-roblugar ?lugar)
(En-objcaja ?obj) (Tiene-caja ?robot) (En-roblugar
?lugar))
  :effect (and (En-objlugar ?obj ?lugar)
(not(En-objcaja ?obj)))
)
)
```

Problema

```
(define (problem TP6_EJ6_prob)
  (:domain TP6_EJ6_dom)
  (:objects oficina cafeteria buzon ROBOT cafe
carta caja)
  (:init
    (lugar oficina)
    (lugar cafeteria)
    (lugar buzon)
    (robot ROBOT)
    (objeto cafe)
    (objeto carta)
    (objeto caja)
    (En-roblugar oficina)
    (En-objlugar cafe cafeteria)
    (En-objlugar carta buzon)
    (En-objlugar caja oficina)
```

```
)  
(:goal  
  (and  
    (En-objlugar cafe oficina)  
    (En-objlugar carta oficina)  
  )  
)  
)
```

Plan obtenido

```
(tomar-caja caja oficina robot)  
(mover oficina buzon)  
(poner-objeto-en-caja carta buzon robot)  
(mover buzon cafeteria)  
(poner-objeto-en-caja cafe cafeteria robot)  
(mover cafeteria oficina)  
(sacar-objeto-de-caja cafe oficina robot)  
(sacar-objeto-de-caja carta oficina robot)  
; cost = 8 (unit cost)
```

Conclusión

El problema se resolvió correctamente modelando el dominio y el problema en PDDL. El planificador genera un plan válido en el que el robot transporta tanto la carta como el café a la oficina utilizando la caja.

Cabe señalar que, si bien la consigna sugería que el robot recogiera la carta en el camino de regreso después de buscar el café, se decidió no forzar esa restricción mediante precondiciones adicionales. Esto se hizo para no restar generalidad al modelo: imponer que el robot solo pueda recoger la carta después del café limitaría la flexibilidad de los planes posibles.

De este modo, el dominio y problema diseñados permiten una solución más general que cumple con los objetivos de planificación sin condicionar el orden exacto de recogida.

w